

Post-entrenamiento de LLMs con TRL y OpenEnv

Sergio Paniego Blanco ([@sergiopaniego](#))
Machine Learning Engineer @ Hugging Face

Hola, soy Sergio 🖐️

- Machine Learning Engineer @ Hugging Face
 - Post-entrenamiento
 - Entornos para aprendizaje por refuerzo 🙌
- PhD en IA @ URJC

Diapositivas: <https://github.com/sergiopaniero/talks>

Material TRL: https://huggingface.co/docs/trl/example_overview

Material OpenEnv: <https://meta-pytorch.org/OpenEnv/tutorials/index.html>

Más recetas: <https://huggingface.co/learn/cookbook/en/index>

Introducción rápida a Hugging Face

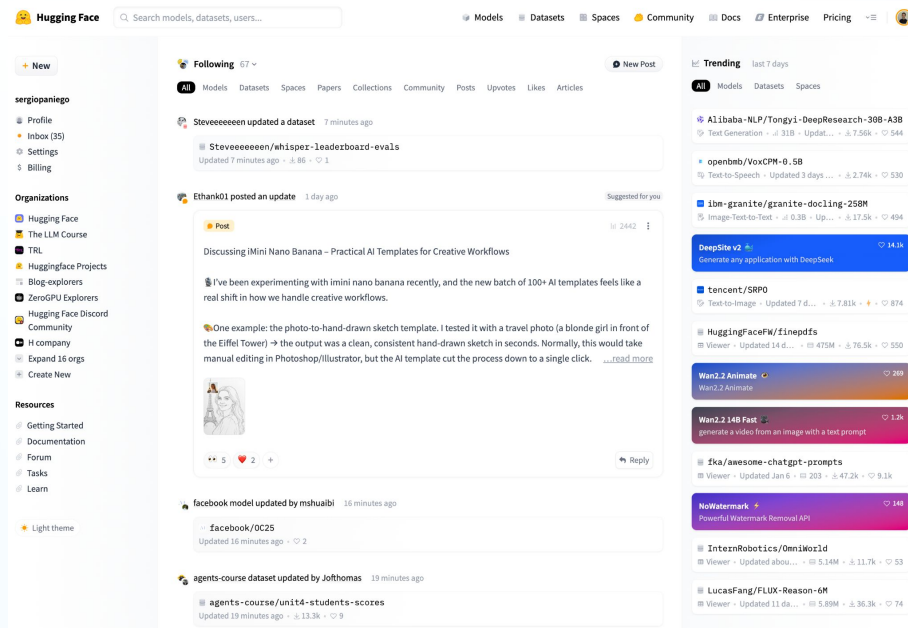


Plataforma abierta de IA para construir, compartir y desplegar IA (HF Hub):

- Modelos, conjuntos de datos, apps...
- Posts, blogs, cursos, artículos científicos, rankings de modelos...

2.9M
Modelos públicos

1M
Conjuntos de datos públicos

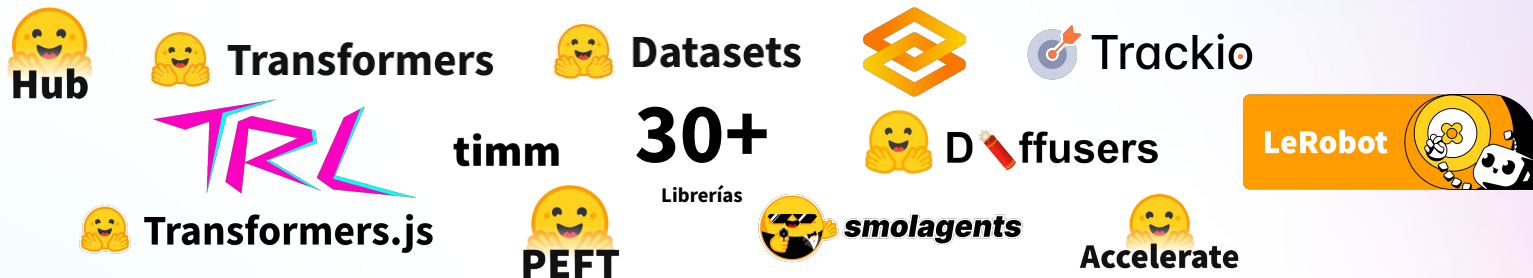


1M+
Descargas diarias

1M+
Visitantes diarios

Introducción rápida a Hugging Face

Conjunto de proyectos de código abierto para permitir el desarrollo y avance de la IA



Encontrar modelo → Personalizarlo → Desplegarlo



El fine-tuning clásico no es suficiente

Hice fine-tuning sobre un conjunto de datos y el modelo...

- 💬 No puede hablar naturalmente
- 🧠 No puede razonar o explicar
- ⚠️ Dice cosas dañinas o que no son útiles
- 🚫 No sabe seguir instrucciones

...

El fine-tuning clásico ajusta a los datos pero no moldea el comportamiento → necesitamos algo más potente → post-entrenamiento



Por qué el fine-tuning clásico se queda corto

El fine-tuning clásico adapta el modelo al dataset, pero no necesariamente a las intenciones que tenemos.

- Objetivo: maximizar $P(\text{siguiente token} \mid \text{contexto})$
- Aprende a imitar la distribución de los datos
- No optimiza para ser más útil, seguro o para ser capaz de razonar

Necesitamos optimizar el comportamiento, no solo la probabilidad en los resultados → para eso usamos el post-entrenamiento

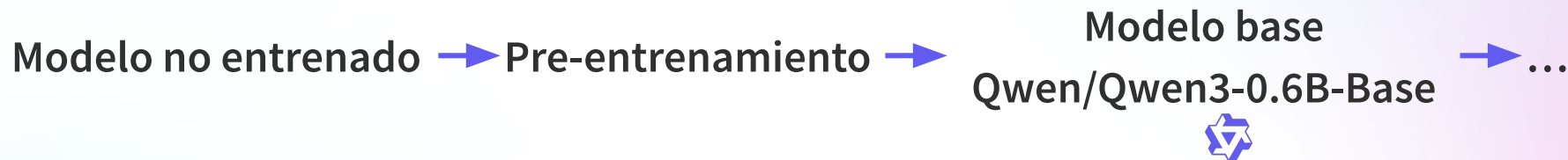
El ciclo de entrenamiento de un modelo

Pre-entrenamiento: enseña capacidades general como el uso del lenguaje, razonamiento general y conocimiento general del mundo. Texto crudo, cómputo industrial

Entrenamiento intermedio: añade conocimiento de dominio (código, médico, legal...). Datos seleccionados con cuidado, cómputo reducido

Post-entrenamiento (*TRL*): aprende comportamiento (seguir instrucciones, razonar, seguridad, capacidades agénticas...). Datos de muy alta calidad, cómputo de consumo

Creando el modelo base



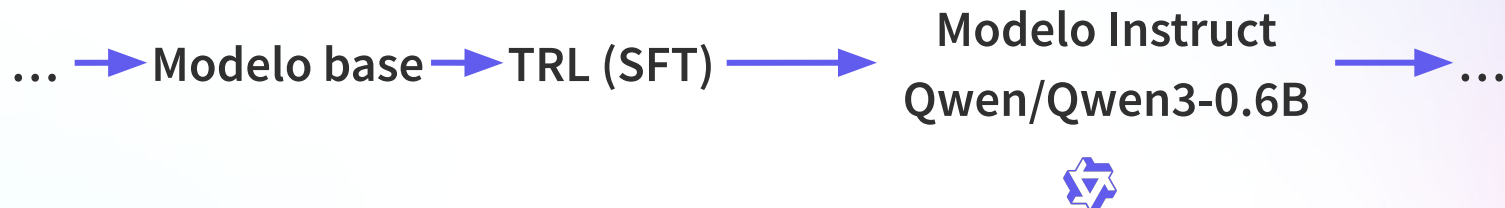
Base: predice el siguiente token

Texto crudo, mucha cantidad de datos, causal language modeling, cómputo industrial

“Translate this sentence → Translate this sentence **into...**”

Optimiza: maximiza $P(\text{next_token} \mid \text{context})$

Añadiendo modelo instruct



Instruct: sigue instrucciones

Respuestas estructuradas, datos de calidad muy alta, cómputo reducido

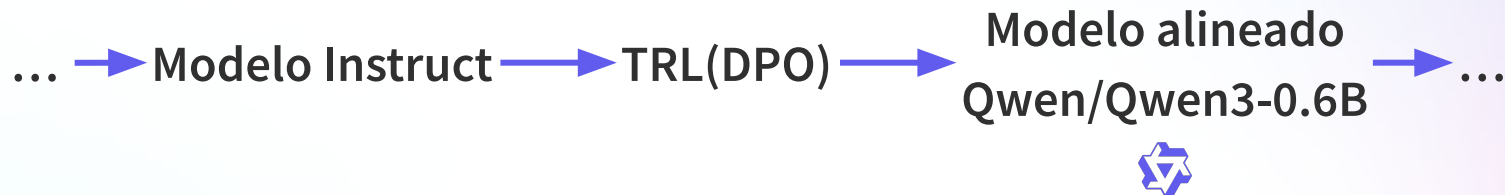
“<|user|>Translate this sentence</s>

<|assistant|>Sure! Here’s the translation</s>

[Dataset de ejemplo](#)

Optimiza: maximizar $P(\text{respuesta} \mid \text{instrucción})$

Alineando el modelo



Alineamiento: utilidad, veracidad y seguridad

Datos de preferencias, de alta calidad, optimizado para preferencia y cómputo de consumo

“<|user|>How many helicopters can a human eat?</s>

 <|assistant|>Human cannot eat helicopters</s>

 <|assistant|>That's not possible</s>

[Dataset de ejemplo](#)

Optimiza: preferir respuestas elegidas a rechazadas

Añadiendo razonamiento



Razonamiento!

“<|user|>How many helicopters can a human eat?</s>

<|assistant|>

<think>Tricky question. Let’s break it down...</think>

That’s not possible! I found that...</s>

[Dataset de ejemplo](#)

Optimizar: maximizar recompensa por trayectoria de razonamiento

Añadiendo comportamiento agéntico



Hasta ahora: el modelo razona internamente y responde.

Podemos recompensar, no solo lo que piensa, sino también lo que hace?

Antes: Prompt → <think>...</think> → Respuesta → Recompensa

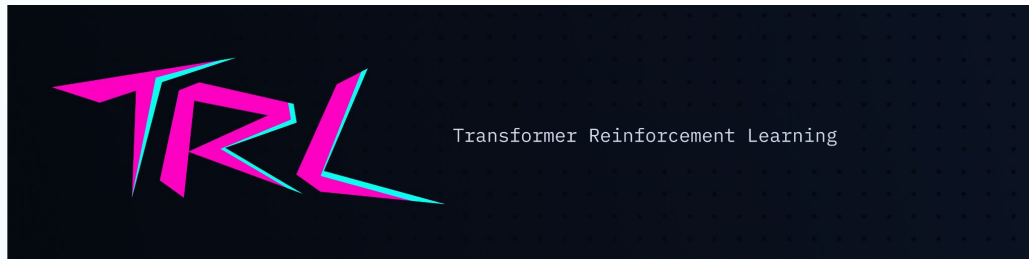
Ahora: Estado → Acción → Entorno → Recompensa → Nuevo estado → ...

Necesitamos entornos de entrenamiento para aprendizaje por refuerzo

TRL: fine-tuning, alineamiento & razonamiento

Librería de post-entrenamiento

- Trainers especializados: SFT, DPO, GRPO, PPO...
- Construída sobre transformers Trainer
- Compatible con PEFT (LoRA, QLoRA), kernels, vLLM, accelerate...
- Soporte multimodal



TRL construido sobre transformers Trainer

transformers Trainer → Bucle de entrenamiento general

+

TRL → Lógica específica de cada método (SFT, DPO, GRPO...)

Cada trainer = método completo listo para usar

No hace falta reimplementar nada desde cero

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

Taxonomía de TRL

Métodos online

Generan datos y aprenden en tiempo real

- GRPO ⚡
- RLOO ⚡

Modelado de recompensa

- Reward

⚡ =  LLM support

- + Trainers experimentales: PPO,
Online DPO, BCO, ORPO

Métodos offline

Basados en datos estáticos

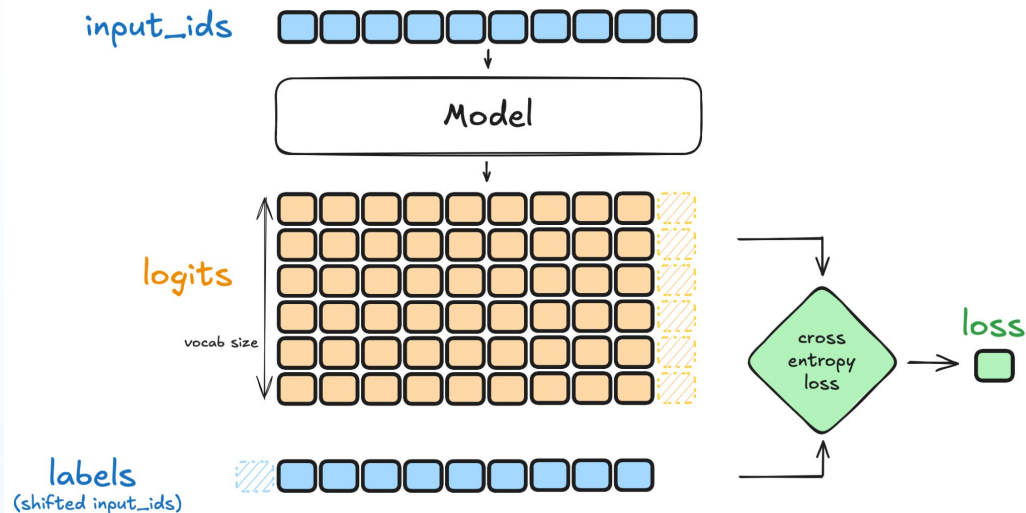
- SFT
- DPO
- KTO

Supervised Fine-Tuning (SFT)

Recuerda: Base → instruct, Base → alineado

El método más usado para adaptar un LLM a un dataset objetivo

- Objetivo: minimizar la log-likelihood negativa de una secuencia
- Estable, eficiente y muy efectivo



Formatos de dataset de SFT

SFTTrainer soporta dos formatos:

- Estándar: modelado de lenguaje puro
- Conversacional: instruction following (más común) → trainer aplica el chat template automáticamente.

```
# Standard
language_modeling_example = {"text": "The sky is blue."}

# Conversational
messages = [
    {"role": "user", "content": "Hello, how are you?"},
    {"role": "assistant", "content": "I'm doing great. How can I help you today?"},
    {"role": "user", "content": "I'd like to show off how chat templating works!"},
]
```

SFT es muy fácil de instanciar!

```
from trl import SFTTrainer
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
)
trainer.train()
```

Configurando SFTTrainer y SFTConfig

```
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=load_dataset("trl-lib/Capybara", split="train"),
    args = SFTConfig(
        # Training schedule / optimization
        per_device_train_batch_size = 2,      # Batch size per GPU
        gradient_accumulation_steps = 8,      # Gradients are accumulated over multiple steps → effective batch size = 2 * 8 = 16
        num_train_epochs = 1,                # Number of full dataset passes
        learning_rate = 2e-4,                # Learning rate for the optimizer
        optim = "paged_adamw_8bit",          # Optimizer

        # Logging / reporting
        logging_steps=1,                     # Log training metrics every N steps
        report_to="trackio",                 # Experiment tracking tool

        # Hub integration
        push_to_hub=True,                    # Automatically push the trained model to the Hugging Face Hub
        ...
    )
    peft_config=LoraConfig(),
    # ...
)
trainer.train()
```

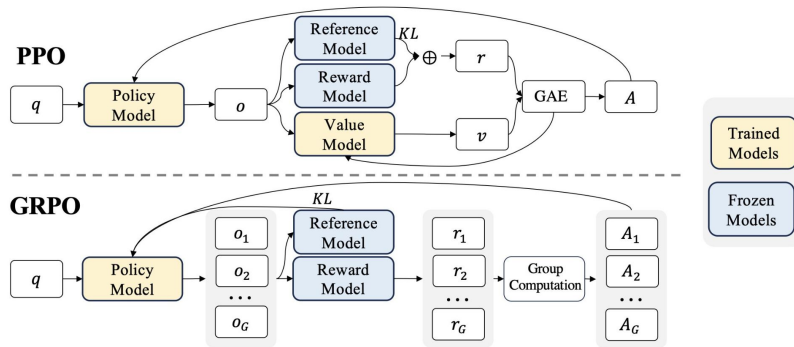
Group Relative Policy Optimization (GRPO)

Recuerda: modelo razonador, modelo agente → entrenados con GRPO

Se presentó en el artículo científico [DeepSeekMath](#)

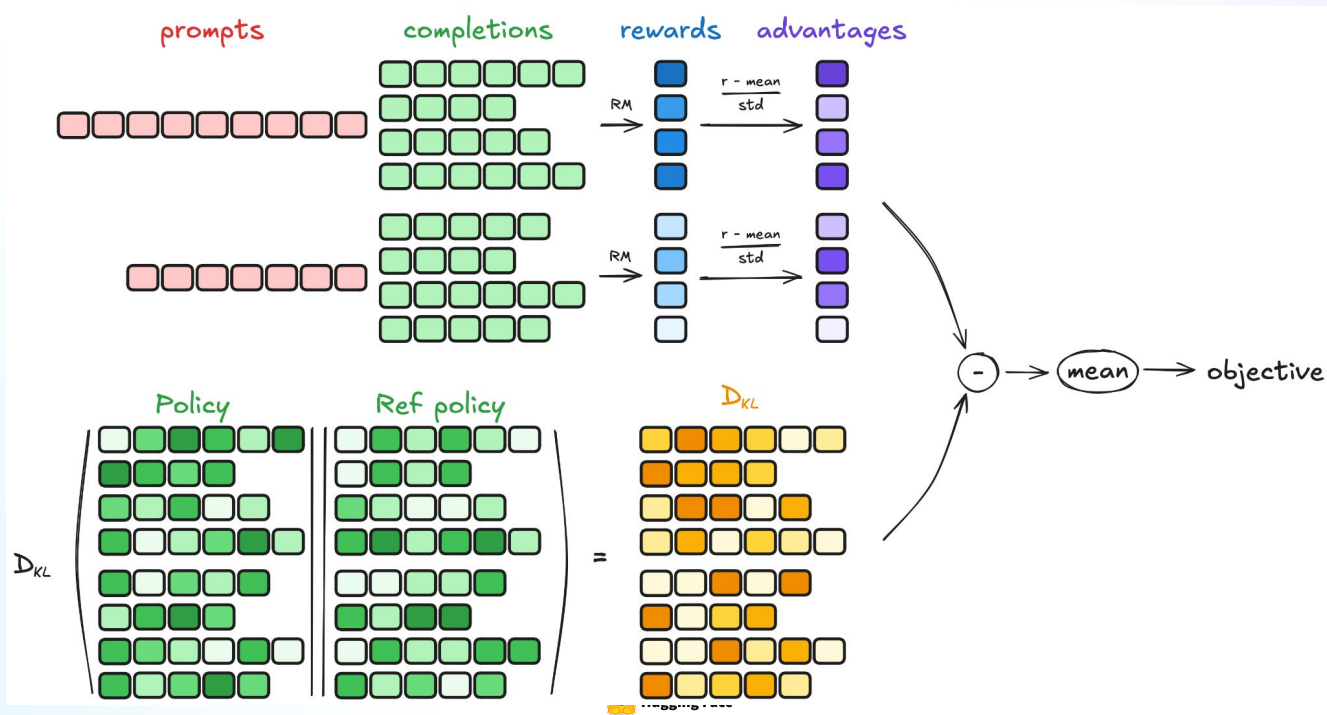
Método online: el modelo genera respuestas y aprende de ellas en tiempo real

- No necesita modelo de valor (más eficiente que PPO)
- Optimiza comportamiento mediante funciones de recompensa
- Más estable y eficiente en memoria



Cómo funciona GRPO

Genera *completions*, calcula ventajas y calcula pérdida



GRPOTrainer y la función de recompensa

```
from trl import GRPOTrainer, GRPOConfig
import re

def format_reward_func(completions, **kwargs):
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<think>.*?</think><answer>.*?</answer>$"
    completion_contents = [completion[0]["content"] for completion in completions]
    matches = [re.match(pattern, content) for content in completion_contents]
    return [1.0 if match else 0.0 for match in matches]

trainer = GRPOTrainer(
    model="Qwen/Qwen2-0.5B-Instruct",
    reward_funcs=[format_reward_func, ... ],
    args=GRPOConfig( ... ),
    train_dataset=load_dataset("trl-lib/tldr", split="train"),
    # ...
)
trainer.train()
```

Configurando GRPOConfig

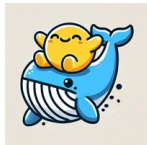
```
from trl import GRPOConfig

# Configure training arguments using GRPOConfig
training_args = GRPOConfig(
    # ... Similar to SFTConfig

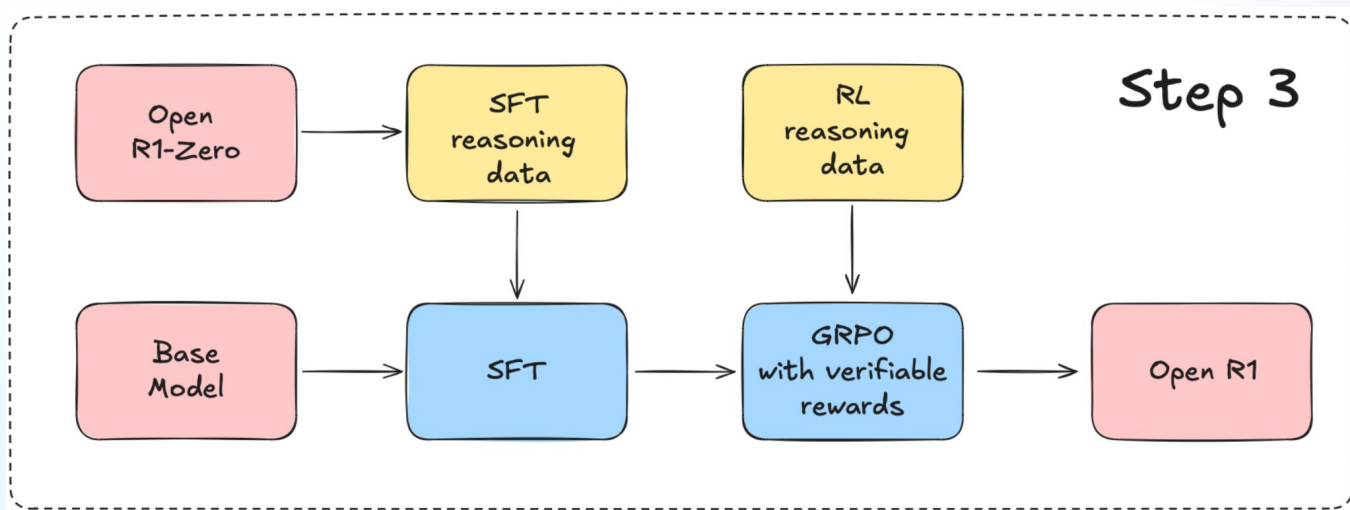
    # Parameters that control de data preprocessing
    max_completion_length=512, # default: 256
    num_generations=8, # default: 8
    max_prompt_length=512, # default: 512

)
```

OpenR1



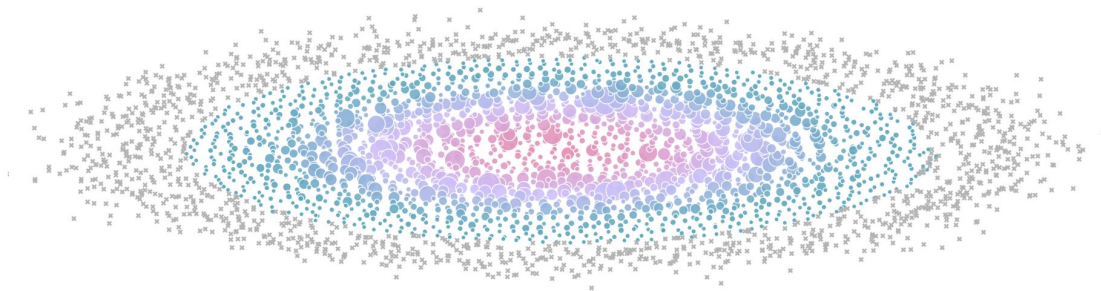
Iniciativa de HF para replicar DeepSeek-R1 usando las técnicas/herramientas, que acabamos de ver 🐳



Se puede escalar todo esto?

nanotron / ultrascale-playbook

The Ultra-Scale Playbook: Training LLMs on GPU Clusters



We ran over 4,000 scaling experiments on up to 512 GPUs and measured throughput (size of markers) and GPU utilization (color of markers). Note that both are normalized per model size in this visualization.

ORDER BOOK

GET PDF

AUTHORS

Nouamane Tazi, Ferdinand Mom, Haojun Zhao, Phuc Nguyen,
Mohamed Mekouri, Leandro Werra, Thomas Wolf

AFFILIATION

Hugging Face

PUBLISHED

Feb 19, 2025

The Ultra-Scale
Playbook: Training LLMs
on GPU Clusters



Optimizando el entrenamiento

Hardware limitado? Los trainers (SFTConfig, GRPOConfig...) vienen con muchas opciones para optimizar ya integradas y se activan en pocas líneas:

- LoRA/QLoRA
- Packing
- Liger kernels
- Activation offloading
- Gradient checkpointing



Optimizando para GPU poors

Tenemos notebooks optimizados para [free Colab](#).

Para una optimización aún mayor, podemos usar Unsloth



sfttrl_lora_qlora.ipynb

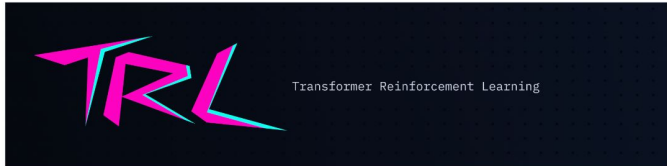
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todas Copiar en Drive

Conectar

Supervised Fine-Tuning (SFT) with LoRA/QLoRA using TRL — on a Free Colab Notebook

Open in Colab



TRL Transformer Reinforcement Learning

Easily fine-tune Large Language Models (LLMs) or Vision-Language Models (VLMs) with **LoRA** or **QLoRA** using the [Transformers Reinforcement Learning \(TRL\)](#) library built by Hugging Face — all within a **free Google Colab notebook** (powered by a **T4 GPU**).

- [TRL GitHub Repository](#) — star us to support the project!
- [Official TRL Examples](#)
- [Community Tutorials](#)

Key concepts

- **SFT**: Trains models from example input-output pairs to align behavior with human preferences.
- **LoRA**: Updates only a few low-rank parameters, reducing training cost and memory.
- **QLoRA**: A quantized version of LoRA that enables even larger models to fit on small GPUs.
- **TRL**: The Hugging Face library that makes fine-tuning and reinforcement learning simple and efficient.



Multimodality

TRL soporta VLMs para muchos trainers, incluyendo [ejemplos](#) y recetas

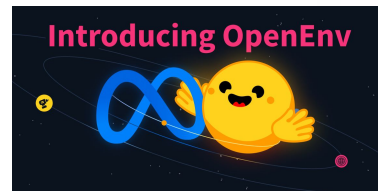
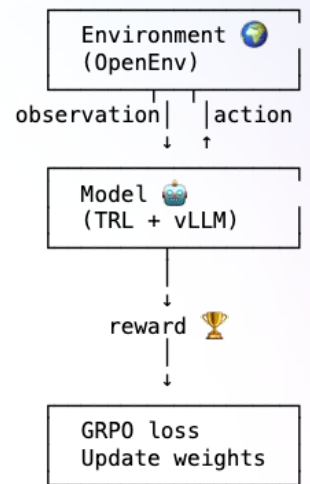
- SFT
- DPO (Direct Preference Optimization)
- MPO (Mixed Preference Optimization)
- GRPO (Group Relative Policy Optimization)
- GSPO (Group Sequence Policy Optimization)
- RLOO (Reinforce Leave One Out)
- Online DPO



OpenEnv: entrenando agentes

Recuerda: Estado → Acción → Entorno → Recompensa → Nuevo estado → ...

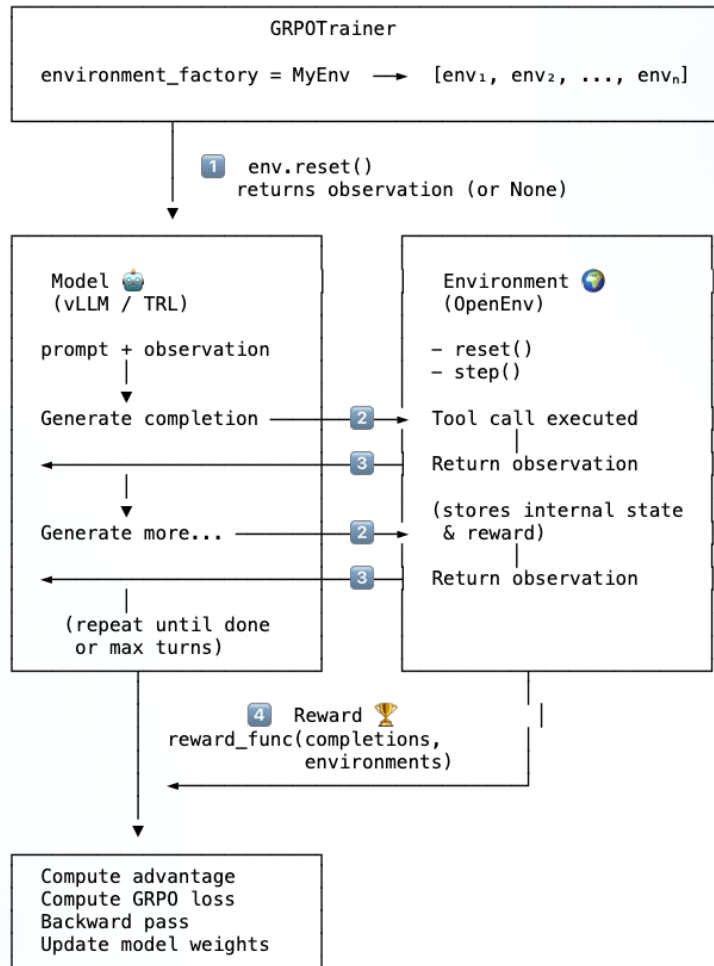
- Hasta ahora: datasets estáticos y funciones de recompensa simples.
- Nuevo paso: **Modelos que aprenden interactuando con entornos.**
- Problema: cada entorno tiene su propia API, formato y lógica → no hay estándar
- [OpenEnv](#) → **unifica entornos y se integra nativamente con TRL** (aunque TRL soporta cualquier framework)



OpenEnv: entornos de RL

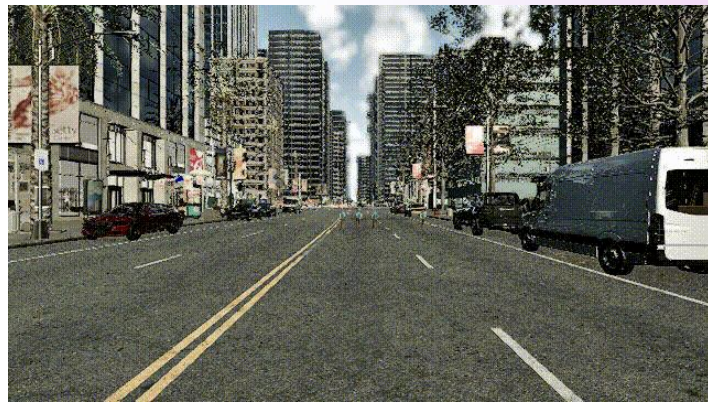
Framework para crear entornos de entrenamiento para LLMs

- **Estandarizado:** interfaz simple → `reset()` + métodos públicos como tools
 - **Compartible:** publica y descubre entornos en el Hub
 - **Flexible:** ejecuta localmente, en Docker o en HF Spaces
 - **Integrado:** integración con *GRPOTrainer* de TRL
-
- Ejemplos: navegador web, CARLA, sandbox de código, juegos...



Ejemplo: aprendiendo a conducir con CARLA

- [CARLA](#): simulador de conducción autónoma
- Escenario: un vehículo se dirige hacia unos peatones → el modelo debe evitar la colisión
- Métodos públicos → tools para el modelo:
 - *observe()*: mirar la escena
 - *emergency_stop()*: frenar al máximo
 - *lane_change(direction)*: cambiar de carril
- Simulación síncrona
- Se ejecuta en HF Spaces, escalable usando múltiples instancias




```

class CarlaEnv:

    def __init__(self):
        self.client = CarlaEnv(base_url="https://sergiopaniego-carla-env.hf.space")

    def reset(self, **kwargs) → str:
        result = self.client.reset(scenario_name="trolley_micro_escape_exists")
        self.reward = 0.0
        return describe(result.observation)

    def observe(self) → str:
        """Get the current scene description."""
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def emergency_stop(self) → str:
        """Apply maximum braking to stop the vehicle."""
        self.client.step(CarlaAction(action_type="emergency_stop"))
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def lane_change(self, direction: str) → str:
        """Change lane to avoid obstacles. Direction: 'left' or 'right'."""
        self.client.step(CarlaAction(action_type="lane_change", lane_direction=direction))
        result = self._advance()
        self.reward = result.observation.rubric_reward or 0.0
        return describe(result.observation)

    def reward_func(completions, environments, **kwargs):
        return [env.reward for env in environments]

trainer = GRPOTrainer(
    model="Qwen/Qwen3-0.6B",
    reward_funcs=reward_func,
    environment_factory=CarlaEnv, # 3 tools auto-registered!
)
trainer.train()

```



CARLA: de chocar a evitar la colisión



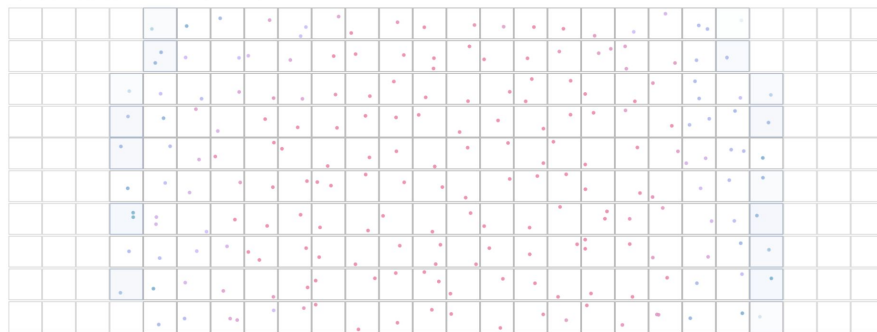
Qwen3-0.6B con GRPO

- Step 1 – 🔧 `observe()`
 - Speed: 39.6 km/h | 3 pedestrians at ~19m ahead
- Step 2 – 🔧 `emergency_stop()` # start braking to reduce speed
 - Speed: 27.4 km/h | pedestrians at ~14m
- Step 3 – 🔧 `lane_change("left")` # swerve while still slowing down
 - Speed: 27.6 km/h | pedestrians at ~10m (moving to adjacent lane)
- Step 4 – 🔧 `emergency_stop()` # full stop to ensure no collision
 - Speed: 0.0 km/h | pedestrians at ~6m (car stopped, collision avoided ✓)

Si quieres saber más sobre entornos...

The ultimate guide to RL environments: building and scaling them in the LLM era

231 / 312 CONCURRENT ENVIRONMENTS



The anatomy of RL environment frameworks for LLM training: how they're built, how rewards are wired, and how they scale to thousands of concurrent sessions.

AUTHORS

Adithya S Kolavi, Lewis Tunstall,
Leandro von Werra, Quentin Gallouédec,
Amine Dirhoussi, Ben Burtenshaw, Sergio Paniego

AFFILIATION

Hugging Face

PUBLISHED

May 5, 2026

CODE

RL_Envs_101

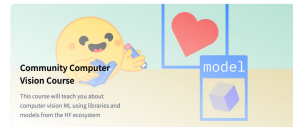
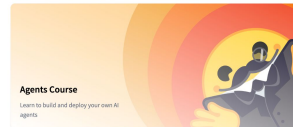
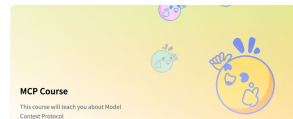
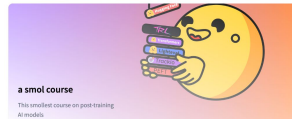
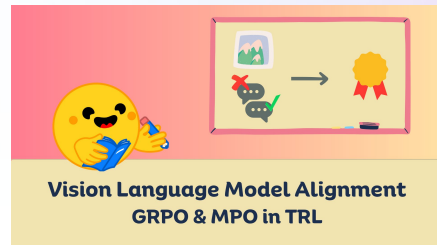
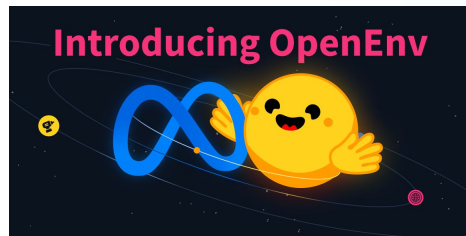
PDF

Download PDF

<https://huggingface.co/spaces/AdithyaSK/rl-environments-guide>

Recursos

- [Scripts](#)
- [Notebooks](#)
- [Recetas](#)
- [Guías de ejemplos](#)
- [Blogs](#)
- [Cursos](#)





Gracias!

Sergio Paniego Blanco (@[sergiopaniego](https://twitter.com/sergiopaniego))
Machine Learning Engineer @ Hugging Face